

2026-06-10-impl-D18-review

D-18 Code Review — `claude_code` / `agentdeck` / `bridle` anti-bluff fixes (§11.4.125)

Reviewer: Independent code-review subagent **Date:** 2026-06-10 **Scope:** Uncommitted D-18 fixes in submodules/`helix_agent/internal/clis/agents/{claude_code,agentdeck,bridle}` / **Disposition:** READ-ONLY review (no edits/commits/push). One temporary mutation applied + restored for §1.1 genuineness; tree verified clean afterward.

Verdict: GO

Build/vet/test: **OK**. Blocking findings: **0**. Non-blocking findings: **2** (informational, both acceptable). Mutation genuineness: **CONFIRMED**. No secrets, scope clean.

1. Build / vet / test (VERIFIED, captured)

```
go build ./internal/clis/agents/{claude_code,agentdeck,bridle}/... → BUI
go vet   ./internal/clis/agents/{claude_code,agentdeck,bridle}/... → VET
go test  -count=1 (3 pkgs, GREEN/RED_MODE unset):
ok  dev.helix.agent/internal/clis/agents/claude_code  3.477s
ok  dev.helix.agent/internal/clis/agents/agentdeck    1.247s
ok  dev.helix.agent/internal/clis/agents/bridle       0.982s
```

2. Anti-bluff substance (VERIFIED — the fixes are HONEST, not relabeled lies)

- **claude_code** `handleMCP` — the hardcoded fleet literal "Available MCP servers: filesystem, github, memory, fetch, puppeteer" and the fabricated "Called %s/%s via MCP" echo are GONE from non-test source. `list` now enumerates `c.mcp.GetServers()` (honest "No MCP servers configured" when empty, sorted real names otherwise). `call` routes through `MCPIntegration.CallTool` → wired `MCPTransport`, and returns the honest wrapped `ErrMCPClientNotWired` when no transport is installed (confirmed in `mcp_integration.go:207-221: if m.transport == nil { return ...ErrMCPClientNotWired }` — no swallowed fabrication; on wired transport the real `*services.ToolCallResult` text flows through `mcpResultText`, `Success = !result.IsError`). This is a genuine real-dispatch seam, not a template.
- **agentdeck** `orchestrate` — "completed" → "planned". Confirmed genuinely honest: the method only calls `createOrchestrationPlan(task, mode)` and returns the plan; **no agent is executed**, so "planned" accurately describes the state. The real plan is still returned.
- **bridle** `runWorkflow` — "completed" → "evaluated"/"blocked". Confirmed honest: the loop only calls `checkGuardrails(step)`; `step.Action` is **never**

dispatched (no action runner exists). Per-step is "evaluated" (or "blocked" on a strict-mode guardrail violation); overall is "blocked" if any step blocked, else "evaluated". The strict-mode blocking path is preserved.

3. Mutation genuineness (VERIFIED — not a tautology)

Reverted bridle "evaluated" → "completed" (both per-step and workflow status) → TestPin_RunWorkflow_NotFabricatedCompleted **FAILED** as required:

expected: "evaluated" actual: "completed" (GREEN guard correctly trips)

Restored → guard passes (ok ... bridle 0.507s); `git diff --stat` back to the original D-18 17-insert/3-delete state. No residue.

Additional cross-check (RED_MODE=1 vs the FIXED artifact, §11.4.115 polarity): every RED assertion across all three packages FAILs because the defect is genuinely absent (e.g. agentdeck RED expects completed, gets planned; claude_code call RED expects the fabricated echo, gets the honest ErrMCPClientNotWired). This proves the guards track real behavior, not a self-agreeing fix.

4. §11.4.120 reconciliation (VERIFIED — N/A, nothing weakened)

No pre-existing test anywhere in `internal/clis/agents/` asserted the old fabricated strings (`Available MCP servers / Called .. via MCP / fabricated completed`) — the only match is a doc-comment in the fixed source. No gate/test was weakened, deleted, or fake-passed; the new pin tests are pure additions.

5. Scope + secrets (VERIFIED)

Changed: exactly the 3 package .go files + 3 new *_pin_test.go. No go.mod/go.sum, no instance_manager, no helix_code, no other package touched. Secret scan of the diff: clean. Mutation-marker scan of the 3 dirs: clean (§11.4.84).

Non-blocking findings (informational, acceptable — do NOT block GO)

- **F1 (nit, claude_code.go Initialize):** when `MCPConfigPath != ""`, `c.mcp = NewMCPIntegration(...)` is constructed twice (the `if c.mcp == nil` branch then the unconditional `if c.config.MCPConfigPath != ""` branch reassigns). Harmless — the second assignment wins and `LoadConfig()` runs on it — but the first construction is dead in that path. Cosmetic only; behavior is correct.
 - **F2 (observation):** the production wiring of a real `MCPTransport` via `SetTransport` is not exercised by these three packages (the stub lives in the unit test per CONST-050(A)). The fix correctly surfaces `ErrMCPClientNotWired` until a transport is installed — honest, not a bluff — but end-to-end MCP dispatch against a real server remains a separate integration concern, not in this batch's scope.
-

Summary line

VERDICT: GO — build OK, vet OK, test OK (3/3 GREEN). Blocking findings: 0. Non-blocking findings: 2 (F1 cosmetic double-construct, F2 scope note). Mutation genuineness CONFIRMED (bridle evaluated→completed → guard FAILs; restored clean). RED-polarity confirmed against fixed artifact. §11.4.120 N/A (no weakened tests). Scope clean (3 pkg dirs only; no go.mod/instance_manager/helix_code), no secrets, no mutation residue.